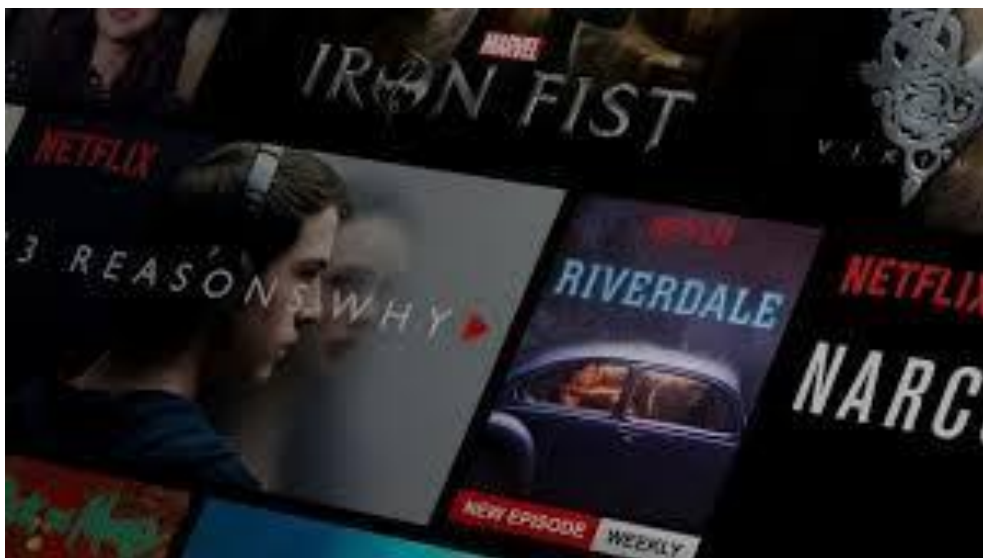


Ano Letivo: 2025/2026

Automação e Desenvolvimento de Software

Movie Recommendation System



Trabalho realizado por:

Martim Antunes **nº:** 2025191528

Daniel Antunes **nº:** 2025192233

Gonçalo Gaio **nº:** 2022224905

Bernardo Pedro **nº:** 2021231014

Francisco Tavares **nº:** 2020211848

Índice

1.Vision and Scope	2
1.1 Problem Statement	2
1.2 Users.....	2
1.3 Vision Statement	2
1.4 List of Features	2
2.Functional Requirements	4
2.1. Functional Requirements - Tradicional Development	4
2.2. Functional Requirements - AI/LLM- Assisted Development	6
2.2.1. Totally Generated by AI.....	6
2.2.2. Human Functional Requirements Enhanced with AI	9
3.1. Non-Functional Requirements - Tradicional Development	13
3.2. Functional Requirements - AI/LLM- Assisted Development	14
3.2.1. Totally Generated by AI.....	14
3.2.2. Human Non Functional Requirements Enhanced with AI.....	16
4. Use Case Diagram.....	17
5. Domain Model.....	18
6. Architecture.....	19
7. User Stories	20
8. Conclusion	36

1.Vision and Scope

1.1 Problem Statement

Modern streaming platforms and online movie databases offer massive collections of films, but users often struggle to discover content that matches their personal preferences. Without intelligent recommendation systems, users spend excessive time browsing through irrelevant titles. This project aims to create a movie recommendation system that helps users quickly find films aligned with their interests by combining traditional development methods with AI-assisted workflows and DevOps practices.

1.2 Users

- General users interested in discovering and organizing movies to watch;
- Movie enthusiasts who enjoy rating and reviewing films;
- Casual viewers who want quick, personalized recommendations;

1.3 Vision Statement

The goal of this project is to build an intuitive and intelligent Movie Recommendation System that allows users to easily discover new movies, rate and receive personalized suggestions.

1.4 List of Features

User Registration and Authentication:

- Create and manage user accounts (sign up, login, logout, change password).

Profile Management:

- View and edit user information.

Movie Catalog Browsing:

- Display paginated lists of movies with key details.

Search and Filter:

- Search by title, director, or genre; filter by rating or year.

Movie Details View:

- Show detailed information about selected movies.

User Ratings:

- Allow users to rate movies (1–5 stars) and update or delete ratings.

Recommendation Engine:

- Suggest movies based on user history and preferences.
- Show “Popular Movies” based on overall ratings.

Watch/Rating History:

- Keep track of movies rated or watched by the user.

My List (Watchlist)

- Allow users to save movies to a personal watchlist for watching later.
- Display saved movies in a dedicated “My List” page.
- Enable adding/removing movies via a heart icon on movie details.

2.Functional Requirements

2.1. Functional Requirements - Tradicional Development

- **FR1 – User Registration (Sign Up)**

The system must allow new users to create an account by providing a unique email address, a password, and a username.

- Passwords must be stored in the database using hashing.
- If the email already exists, registration must be denied with an error message.

- **FR2 - User Login / Logout**

The system must allow registered users to log in with their email and password.

- If login is successful, a session or authentication token must be created.
- The system must provide a logout option to end the session.

- **FR3 – Profile Management**

The system must allow users to view and edit their profile information (display name, email, and password).

- **Email must remain unique.**
- **Password change requires entering the current password.**

- **FR4 – Movie List**

The system must display a catalog of movies with the following details: title, genre, release year, description, director, and average rating.

- **FR5 – Search by Attributes**

The system must allow users to search for movies by title or genre.

- **FR6 – Filters**

The system must allow filtering search results by rating range (e.g., 1–5 stars) and release year.

- **FR7 – Rate a Movie**

The system must allow authenticated users to rate a movie on a scale of 1 to 5 stars.

- **FR8 - Personalized Recommendations (Recommended For You)**

The system must display a “**Recommended For You**” section on the homepage. Recommendations should be personalized based on:

- The user’s rating history.
- Genres similar to the ones previously rated highly by the user.

- **FR9 – Popularity-Based Recommendations**

The system must display a “**Top Movies**” section showing the most popular movies across all users.

- **FR10 – Watch History**

The system must maintain a record of movies that the user has rated or marked as watched.

- **FR11 – My List (Watchlist)**

The system must allow users to create and manage a personal “**My List**” (or **Watchlist**) to save movies they wish to watch later.

2.2. Functional Requirements - AI/LLM- Assisted Development

2.2.1. Totally Generated by AI

- **FR1 – User Management**

Handles all aspects of user authentication and profile management.

- The system shall allow users to register with email, username, and password.
- The system shall validate user credentials and store passwords securely (hashed).
- The system shall allow users to log in and log out securely.
- The system shall maintain user sessions during navigation.
- The system shall allow users to edit their profile information.
- The system shall provide password reset functionality via email verification.

- **FR2 – Movie Search and Browsing**

Enables users to search and explore movies using different filters and retrieve data from an external API.

- The system shall allow users to search movies by title, genre, actor, or year.
- The system shall fetch movie data (title, poster, description, cast, release date, rating, etc.) from an external API (e.g., TMDB, IMDb, OMDb).
- The system shall display detailed movie information on a movie detail page.
- The system shall allow browsing of trending or popular movies.
- The system shall store basic movie data locally for faster access.

- **FR3 – Movie Rating and Feedback**

Allows users to rate movies and use ratings to improve recommendations.

- The system shall allow users to rate movies they have watched.
- The system shall allow users to edit or delete their ratings.
- The system shall display average ratings for each movie.
- The system shall store user ratings to feed the recommendation engine.

- **FR4 – WatchList**

Lets users manage personalized lists for later and watch movies.

- The system shall allow users to add movies to a “Watch Later” list.
- The system shall allow users to add movies to an “Already Watched” list.
- The system shall allow users to remove movies from these lists.
- The system shall display user watchlists in their profile/dashboard.

- **FR5 – Movie Recommendations**

Provides personalized movie recommendations based on user data and behavior.

- The home page shall display recommended movies. Recommendations shall consider rated movies, watch history, and preferred genres.
- The system shall display recommendations in an attractive grid or carousel layout.
- Recommendations shall update when the user interacts with new movies.

- **FR6 – System Administration (Optional/Future Extension)**

Provides tools for admin users to manage and monitor the system.

- Admins shall be able to manage user accounts (activate/deactivate).
- Admins shall monitor API usage and system performance.
- Admins shall trigger manual synchronization of movie data.

- **FR7 – External API Integration**

Integrates with external movie APIs for content population and updates.

- The system shall fetch movie data from an external API such as TMDB or IMDb.
- The system shall locally store the fetched data for recommendation and quick access.
- The system shall handle API rate limits gracefully.
- The system shall periodically refresh local data.

- **FR8 – Notifications (Optional)**

Keeps users updated about recommendations and new releases.

- The system may notify users of new recommended or released movies.
- The system may send email notifications based on user preferences.

2.2.2. Human Functional Requirements Enhanced with AI

- **FR1 – User Registration (Sign Up)**

The system must allow new users to register for an account with unique credentials.

- Users must provide a unique email, a username, and a password.
- The system must validate that the email is not already in use.
- Passwords must be hashed using a secure algorithm (e.g., bcrypt or Argon2) before being stored.
- Upon successful registration, users should be redirected to the login page or automatically logged in.
- In case of duplicate email or invalid input, a clear error message must be shown.

- **FR2 - User Login / Logout**

The system must provide secure user authentication and session management.

- Users log in using their email and password.
- If authentication is successful, the system must issue a JWT token or start a session.
- Invalid credentials must result in an appropriate error message.
- A logout option must be available to end the user session and invalidate the token.

- **FR3 – Profile Management**

The system must allow users to manage their profile information securely.

- Users can **view and edit** their display name, email, and password.
- Email addresses must remain **unique** across all users.
- Changing the password requires the user to enter their **current password**.

- The system must confirm successful updates and handle validation errors gracefully.

- **FR4 – Movie Catalog**

The system must display a comprehensive catalog of movies sourced from an external API (e.g., IMDb API).

- Each movie entry must show **title, genre, release year, description, director, and average rating**.
- The catalog must support **pagination** or infinite scrolling for large datasets.
- Movie data should be periodically **synchronized** from the external API to ensure freshness.

- **FR5 – Search by Attributes**

The system must allow users to search for movies efficiently.

- Users can search by **movie title** or **genre**.
- The search function must support **partial matches** (e.g., “Aveng” → “Avengers”).
- Search results must display key movie details (title, year, genre, rating).

- **FR6 – Filters**

The system must provide filtering options to refine movie search results.

- Users can filter by **rating range** (e.g., 1–5 stars).
- Users can filter by **release year** or **year range**.
- Filters should be **combinable** with search queries for precise results.

- **FR7 – Rate a Movie**

The system must allow authenticated users to rate movies.

- Only logged-in users can submit a **rating** (1–5 stars).
- Users can **update or remove** their rating at any time.
- The **average rating** displayed for each movie must be dynamically recalculated.

- **FR8 - Personalized Recommendations (Recommended For You)**

The system must generate personalized movie recommendations for each user.

- The homepage must display a “**Recommended For You**” section.
- Recommendations are based on:
 - The user’s **rating history**.
 - **Genres** of movies previously rated highly by the user.
- The recommendation model must be **updated dynamically** as user interactions change.

- **FR9 – Popularity-Based Recommendations (Top Movies)**

The system must display the most popular movies among all users.

- The “**Top Movies**” section must rank films based on:
 - Number of ratings.
 - Average rating score.
- This list should be **automatically updated** as new ratings are submitted.

- **FR10 – Watch History**

The system must maintain a record of movies that users have interacted with.

- Each user's watch history must include movies that were **rated** or marked as **watched**.
- The system should allow users to view their complete watch history.
- Duplicates must be avoided (each movie appears once per user).

- **FR11 – My List (Watchlist)**

The system must allow users to maintain a personal watchlist.

- Users can **add** movies they want to watch later.
- Users can **remove** movies from their list at any time.
- The "My List" section must be easily accessible from the user's profile or navbar.
- A movie can appear in only one state: **watchlist** or **watched** (if applicable).

3. Non-Functional Requirements

3.1. Non-Functional Requirements - Traditional Development

- **NFR1 – Performance**

The system must respond to user requests (e.g., browsing, searching) within **2 seconds** under normal load conditions (≤ 50 concurrent users).

- **NFR2 - Usability**

The web application must provide an intuitive and user-friendly interface, accessible through desktop and mobile browsers.

- **NFR3 - Security**

Password Protection: All user passwords must be hashed using a secure algorithm before being stored in the database.

Authentication: The system must use token-based authentication (e.g., JWT) to protect API requests after login.

Session Management: User sessions/tokens must expire after a configurable period of inactivity (default: 30 minutes).

3.2. Functional Requirements - AI/LLM- Assisted Development

3.2.1. Totally Generated by AI

- **NFR1 – Performance**

Defines response and processing time constraints.

- The system shall respond to user requests (login, search, etc.) within 2 seconds.
- The recommendation updates shall complete within 5 seconds.
- The system shall handle at least 1000 concurrent users efficiently.

- **NFR2 – Scalability**

Ensures the system can scale with user growth.

- The backend shall support horizontal scaling.
- The database shall be optimized for large datasets and support caching.

- **NFR3 – Scalability**

Protects user data and ensures safe interactions.

- All communications shall occur over HTTPS.
- Passwords shall be hashed and salted.
- JWT or session-based authentication shall be used.
- The system shall prevent SQL injection, XSS, and CSRF attacks.
- The system shall comply with GDPR principles for data protection.

- **NFR4 – Usability**

Focuses on user experience and interface design.

- The UI shall be intuitive and responsive on all devices.
- The home page shall highlight personalized recommendations.
- The design shall follow consistent navigation and visual patterns.

- **NFR5 – Reliability and Availability**

Ensures consistent availability and fault tolerance.

- The system shall have 99% uptime.
- The system shall handle external API failures gracefully.
- The system shall recover automatically from minor outages.

- **NFR6 – Maintainability**

Facilitates ease of updates, testing, and future extensions.

- The system shall use modular and well-documented code (MVC architecture).
- The system shall allow easy integration of new APIs.
- The system shall include unit and integration tests.

- **NFR7 – Technology Stack**

Defines the main technologies and environment.

- Frontend: React.js (with hooks or Redux).
- Backend: Java (Spring Boot).
- Database: MySQL / PostgreSQL / MongoDB.
- External API: TMDB / IMDb / OMDb.
- Authentication: JWT / Spring Security.
- Deployment: Docker containers or cloud (AWS, Azure, or GCP).

3.2.2. Human Non Functional Requirements Enhanced with AI

- **NFR1 – Performance**

The system must provide fast and efficient responses under normal operating conditions.

- All user interactions (e.g., browsing, searching, rating) should respond within **2 seconds** under normal load.
- The system must support at least **50 concurrent users** without noticeable degradation.
- APIs should be **optimized** using caching and efficient database queries.

- **NFR2 - Usability**

The system must offer a consistent, intuitive, and responsive user interface.

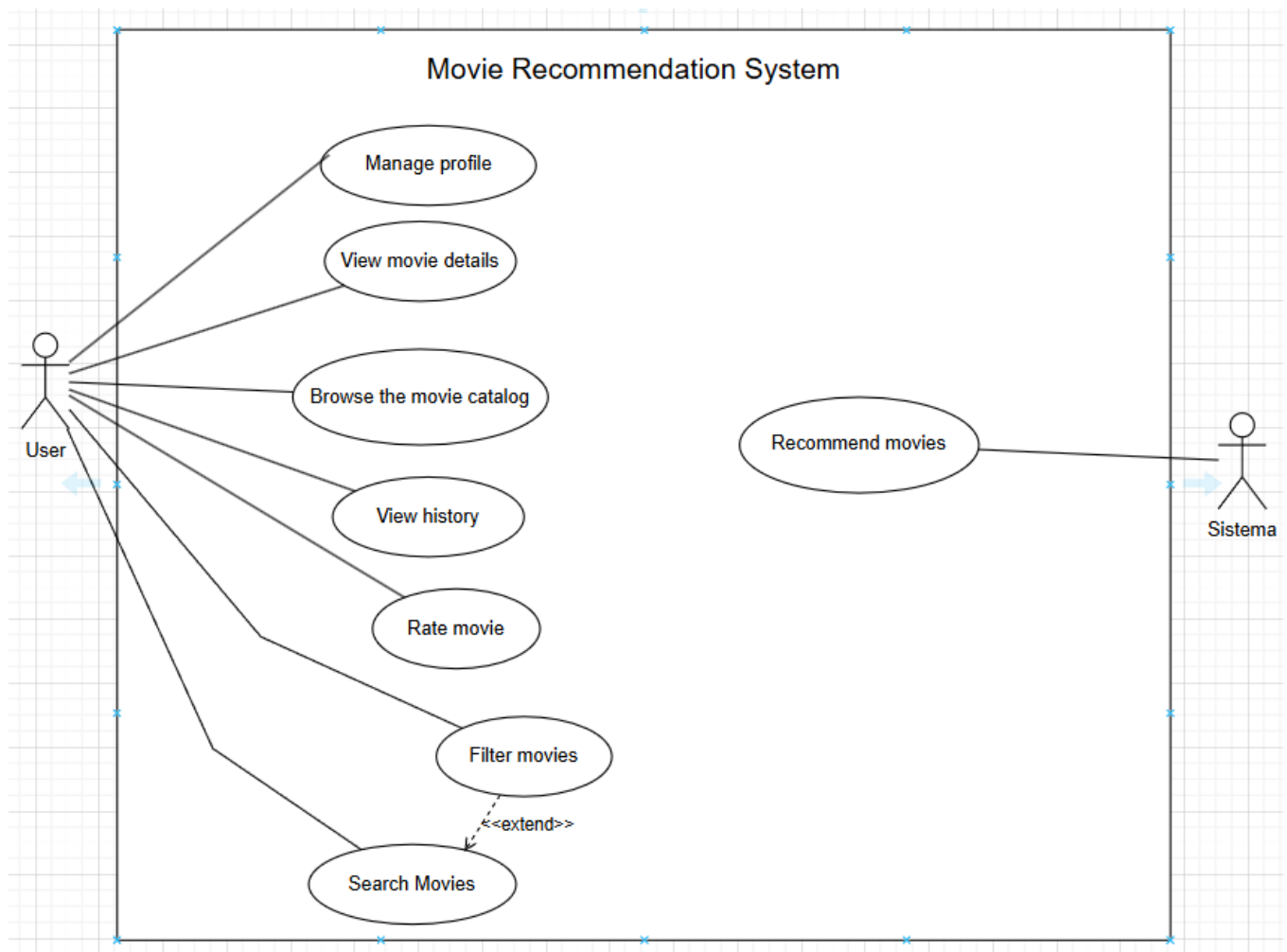
- The web application must be fully **responsive**, functioning seamlessly on both desktop and mobile browsers.
- The UI should follow **modern design principles**, focusing on clarity and accessibility.
- Feedback (e.g., success/error messages, loading indicators) must be provided for user actions.

- **NFR3 - Security**

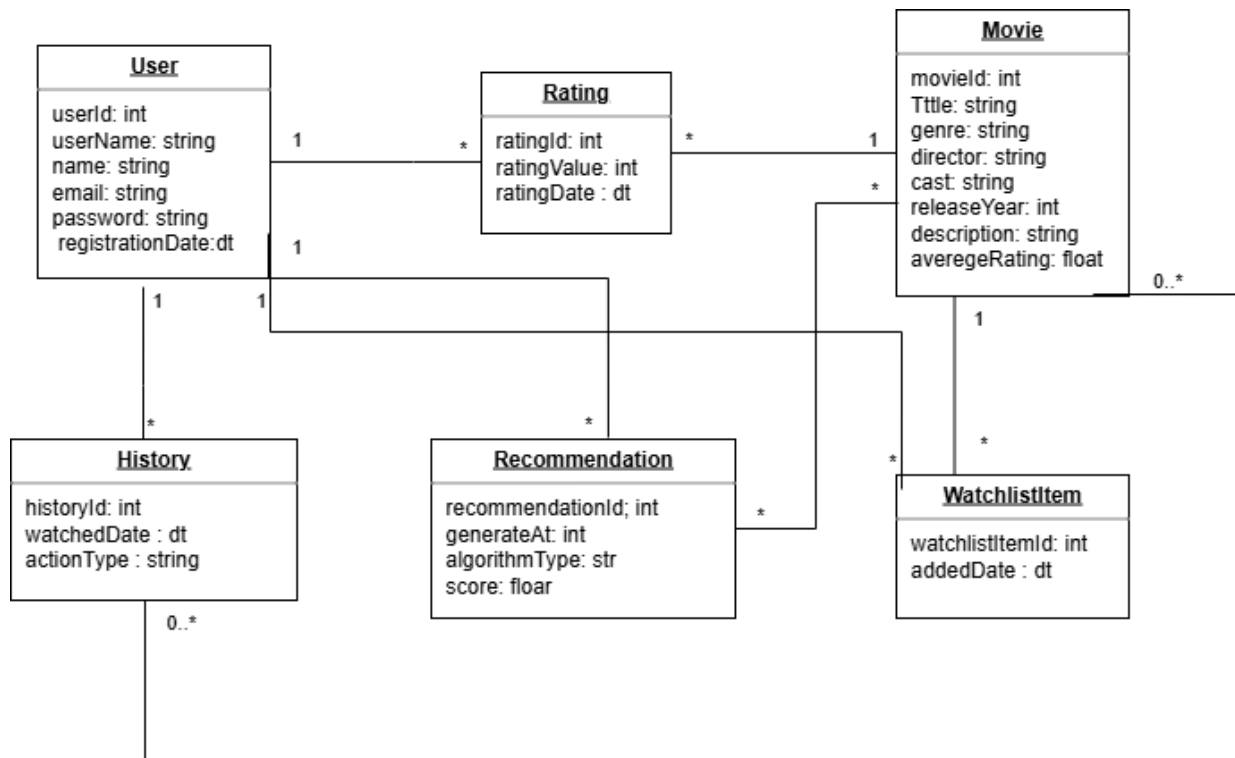
The system must ensure strong protection of user data and sessions.

- **Password Protection:** All passwords must be securely hashed before storage (using bcrypt, Argon2, or equivalent).
- **Authentication:** API requests after login must use **JWT-based authentication**.
- **Session Management:** Tokens must expire after a configurable period of inactivity (default: 30 minutes).
- **Data Protection:** Sensitive data must be transmitted via **HTTPS**.
- **Input Validation:** All inputs must be validated and sanitized to prevent SQL injection and XSS attacks.

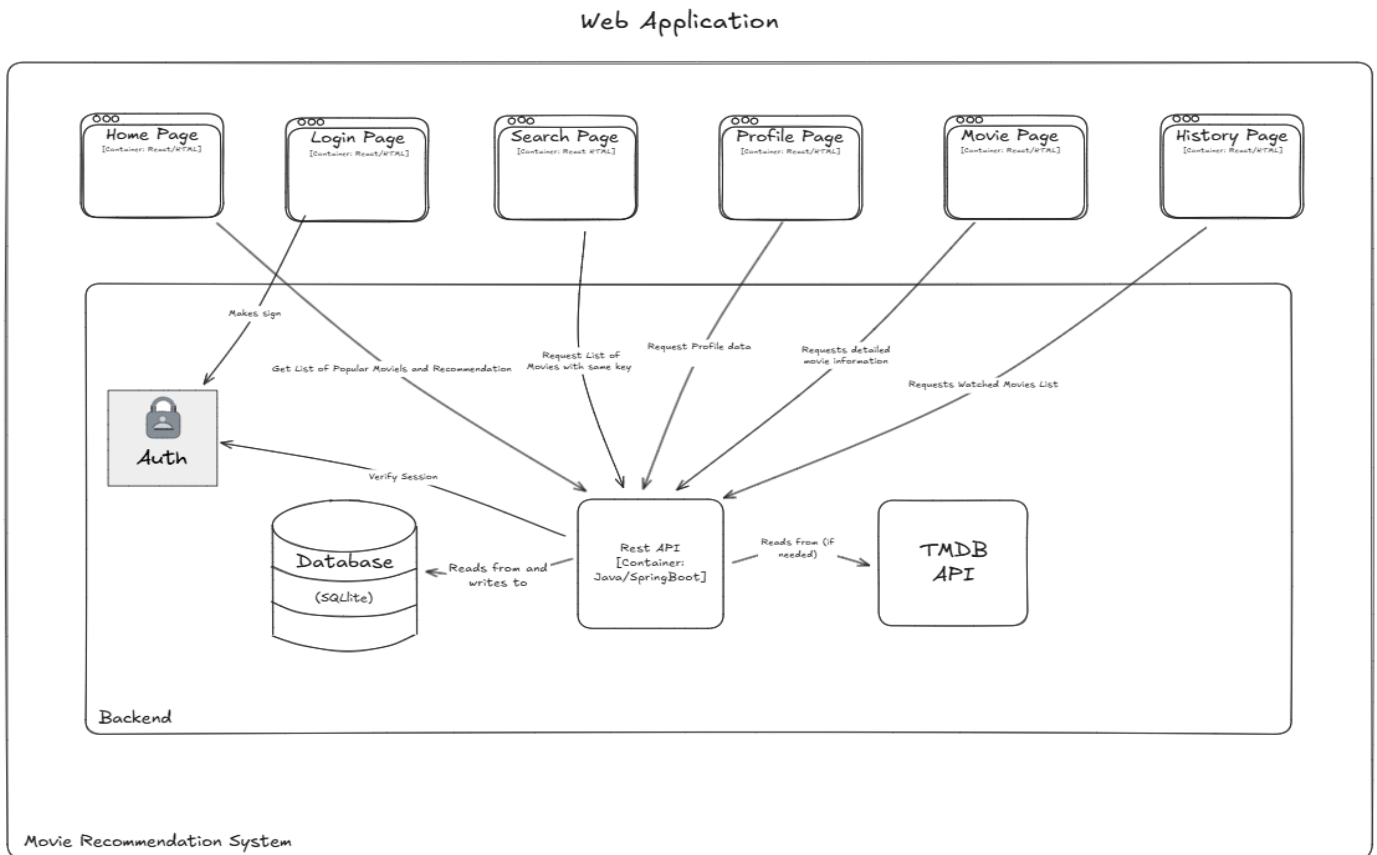
4. Use Case Diagram



5. Domain Model



6. Architecture



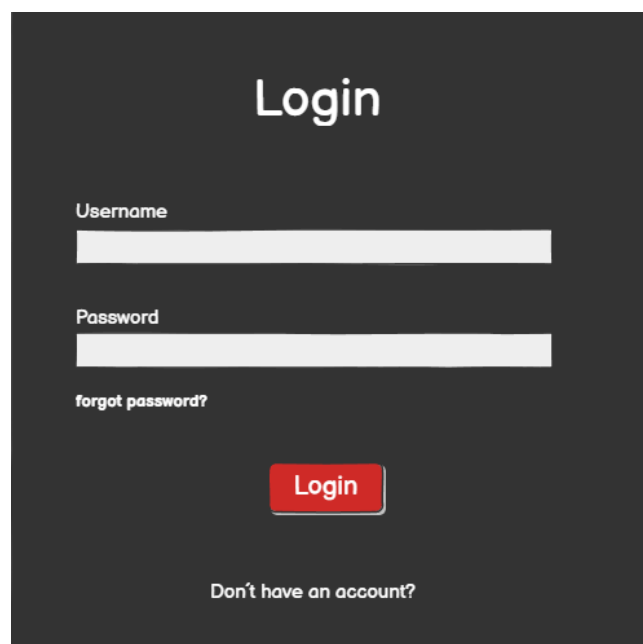
7. User Stories

[US1] Login

As a user, I want to log in using my registered email and password,so that I can access my account and personalized movie recommendations.

How are we going to do this?

- UI Design: Build a Login page with fields for email, password, and a Login button.
- Backend: Create an API endpoint for authentication (verify email + password).
- Database: Validate credentials against stored user records.
- Error Handling: Show error messages for invalid credentials or inactive accounts.
- Testing: Test login with valid and invalid credentials, including edge cases.



Login

Username

Password

[forgot password?](#)

Login

[Don't have an account?](#)

Acceptance Criteria

- Successful Login
Given the user has an existing registered account,
When the user enters the correct email and password,
Then they should be authenticated successfully and redirected to the home page.

- Error on Invalid Credentials

Given the user attempts to log in with an incorrect email or password,

When the user clicks the Login button,

Then a clear error message should be displayed: "Invalid email or password".

- Mandatory Fields Validation

Given the user leaves the email or password fields empty,

When the user clicks the Login button,

Then an error message should appear for each empty field: "Email is required" or "Password is required".

- Password Masking

Given the user is on the login page,

When the user types in the password field,

Then the password characters should be masked (displayed as dots or asterisks).

- Forgot Password Link

Given the user can't remember their password,

When the user clicks the Forgot Password link,

Then they should be redirected to a page or modal where they can request a password reset.

- Access Control After Login

Given the user has logged in successfully,

When they attempt to access restricted pages (e.g., profile, account information, etc.),

Then they should be able to access these pages without needing to log in again.

- Logout

Given the user is logged in

When they click the Logout button,

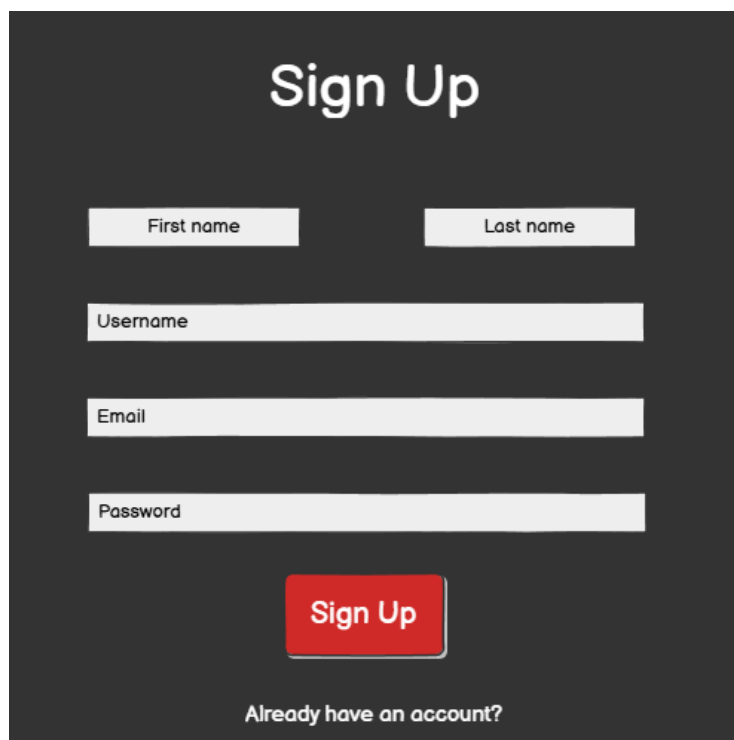
Then their session should be terminated, and they should be redirected to the login page.

[US2] Sign Up

As a user, I want to be able to create an account using my credentials, So that I can access the app and manage my information..

How are we going to do this?

- UI Design: Build a user-friendly Sign Up page with fields for email, password, first name, last name, username and a Sign Up button.
- Backend: Create an API to handle user registration.
- Database: Store new user information securely, including hashed passwords, and ensure that email addresses are unique.
- Error Handling: Handle errors such as duplicate email addresses with clear error messages.
- Testing: Test the registration process for success and various error scenarios.



The image shows a 'Sign Up' form on a dark background. At the top, the title 'Sign Up' is displayed in a large, white, sans-serif font. Below the title, there are five input fields: 'First name' and 'Last name' are side-by-side at the top, followed by 'Username', 'Email', and 'Password' stacked vertically. Each field has a light gray border and placeholder text. At the bottom of the form is a prominent red button with the text 'Sign Up' in white. Below the button, there is a link that says 'Already have an account?' in a smaller white font.

Acceptance Criteria

- **Successful Registration**

Given the user provides a valid email and a strong password,

When the user clicks the Sign Up button,

Then their account should be created successfully, and they should be redirected to the login page with a success message.

- **Error on Duplicate Email**

Given the user attempts to register with an email that is already in use,

When the user clicks the Sign Up button,

Then an error message should be displayed: "Email already exists."

- **Password Strength Validation**

Given the user enters a weak password,

When they click the Sign Up button,

Then an error message should indicate that the password must meet strength criteria (e.g., length, complexity).

- **Mandatory Fields Validation**

Given the user leaves any of the required fields empty,

When the user clicks the Sign Up button,

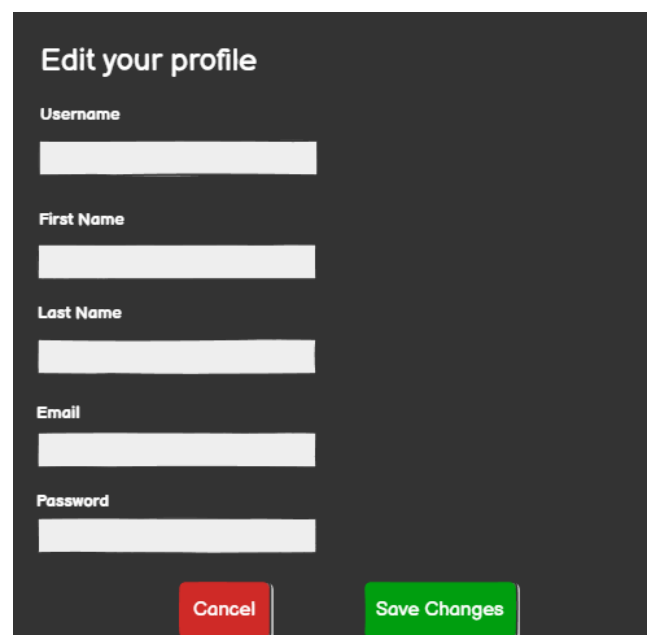
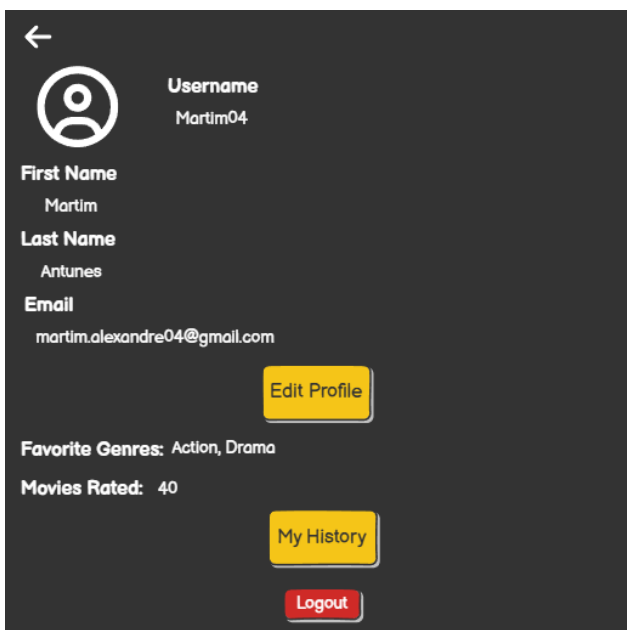
Then an error message should appear for each empty field: "Email is required" or "Password is required."

[US3] Profile Management

As a user, I want to view and edit my profile information, so that I can keep my account details up to date.

How are we going to do this?

- UI Design: Create a Profile page displaying user details (email, username, first name, last name) with an option to edit them.
- Backend: API to fetch profile details of the logged-in user; API to update profile information.
- Database: Retrieve and update user records; Ensure email remains unique.
- Validation: Required fields must not be empty, and new values must follow format rules (valid email, unique username).
- Testing: Verify successful updates, error handling, and persistence of changes.



Acceptance Criteria

- View Profile

Given the user is logged in,

When they navigate to the Profile page,

Then their account details (email, username, first/last name) should be displayed.

- Update Profile

Given the user edits profile fields with valid data,

When they click Save,

Then the changes must be stored and shown on their Profile page.

- Validation Errors

Given the user leaves a required field empty or enters invalid data,

When they click Save,

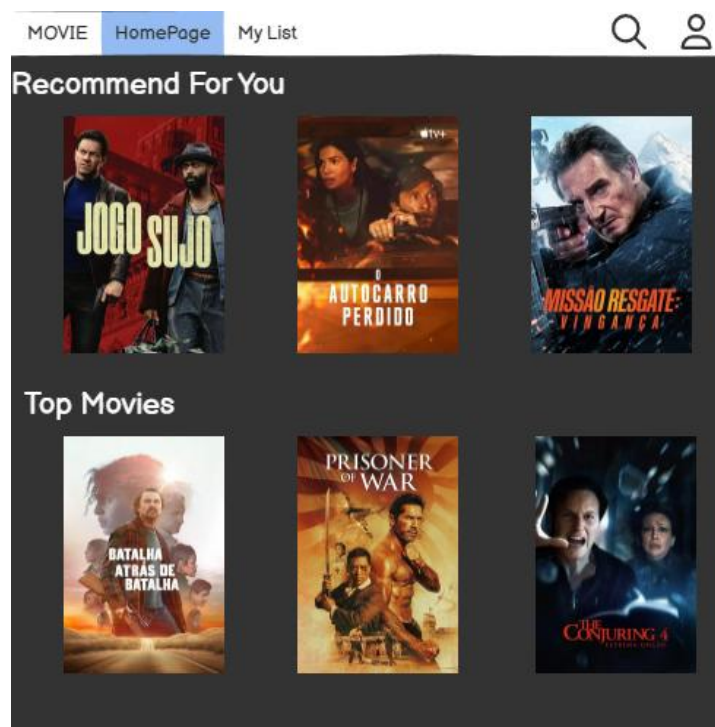
Then error messages must indicate the problem (e.g., “Email already exists” or “Invalid format”).

[US4] Browse Movie Catalog

As a user, I want to browse a catalog of movies, so that I can discover what is available.

How are we going to do this?

- UI Design: Create a catalog page displaying a grid/list of movies with title.
- Backend: API endpoint to return a paginated list of movies.
- Database: Fetch movies from database with metadata.
- Testing: Check correct data display and pagination.



Acceptance Criteria

- Catalog Display

Given movies exist in the database,

When the user opens the catalog page,

Then a list of movies with titles should be displayed.

- Pagination

Given more than N movies exist,

When the user navigates through the catalog,

Then pagination controls should allow them to move between pages.

[US5] Movie Details Page

As a user, I want to view details of a specific movie, so that I can learn more about it before watching or rating.

How are we going to do this?

- UI Design: Add a movie details page with description, cast, director, release year, genre, and ratings.
- Backend: API to fetch full movie details by ID.
- Database: Store extended attributes per movie.
- Testing: Ensure correct details load for each movie.



Acceptance Criteria

- Open Movie Details

Given the user clicks a movie in the catalog,

When the details page loads,

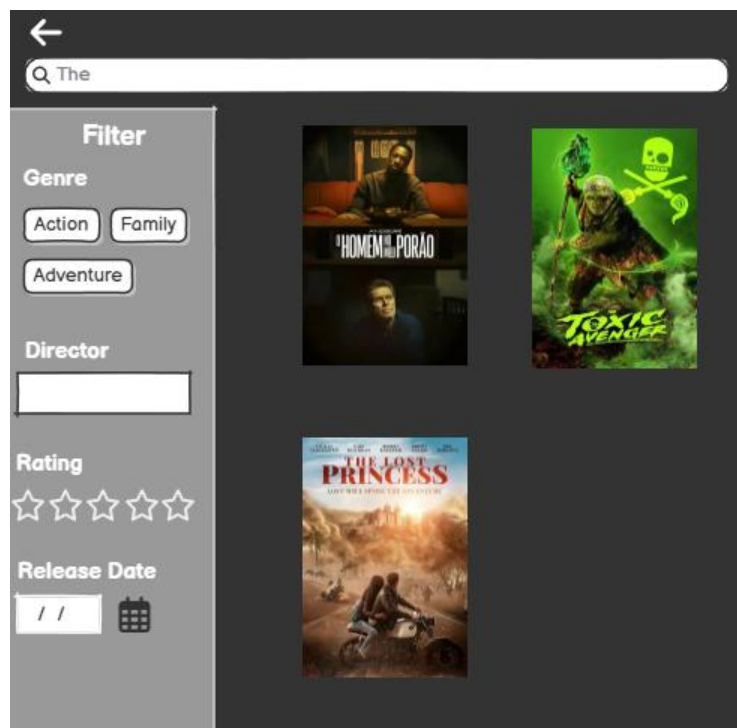
Then the movie's full information (title, genre, director, release year, description, average rating) should be displayed.

[US6] Search Movies

As a user, I want to search movies by title, director or genre, so that I can quickly find what I'm interested in.

How are we going to do this?

- UI Design: Add a search bar on the catalog page.
- Backend: API to support search queries by multiple attributes.
- Database: Use indexed search for efficiency.
- Testing: Test search with exact match, partial match, no results.



Acceptance Criteria

- Successful Search

Given the user enters a keyword in the search bar,

When they submit the search,

Then movies matching the title, director, or genre should be displayed.

- No Matches

Given the user enters a keyword with no matches,

When they submit the search,

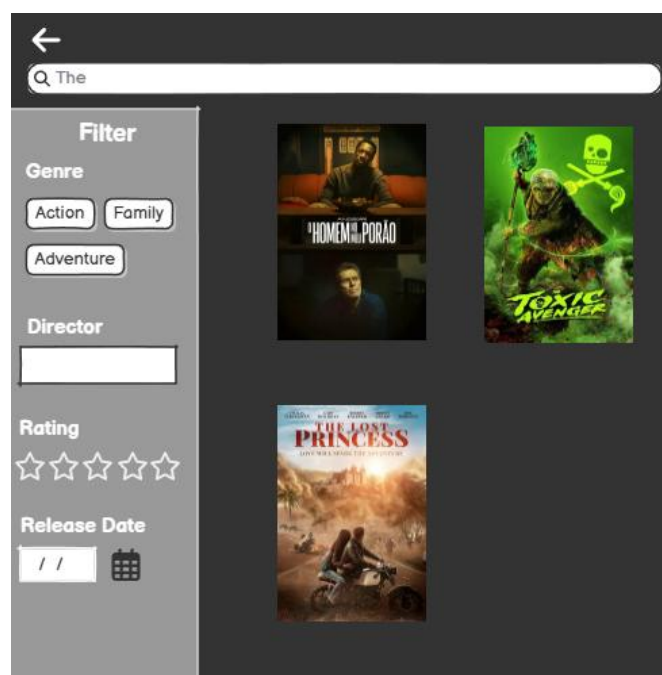
Then the system should display: "No movies found."

[US7] Filter Movies

As a user, I want to filter movies by rating and release year, so that I can narrow down my options.

How are we going to do this?

- UI Design: Add filter controls (dropdowns or sliders) for rating and year.
- Backend: API endpoint supporting query parameters for filters.
- Database: Query movies matching the filter conditions.
- Testing: Ensure filters combine correctly (e.g., genre + rating).



Acceptance Criteria

- Filter by Rating

Given movies exist with various ratings,

When the user selects a rating filter,

Then only movies matching that rating range should be displayed.

- Filter by Release Year

Given movies exist from different years,

When the user selects a release year,

Then only movies from that year should be displayed.

- Filter by Director

Given movies exist directed by different directors,

When the user selects a specific director from the filter options or search input,

Then only movies directed by that person should be displayed.

- Filter by Genre

Given movies belong to different genres,

When the user selects one or more genres from the available filter options,

Then only movies matching the selected genres should be displayed.

[US8] Rate a Movie

As a user, I want to rate a movie from 1 to 5 stars, so that I can share my opinion and improve recommendations.

How are we going to do this?

- UI Design: Add a rating widget (stars) on the movie details page.
- Backend: API to submit ratings for movies.
- Database: Store rating per user per movie.
- Testing: Ensure multiple ratings update average correctly.



Acceptance Criteria

- Successful Rating

Given the user is logged in and hasn't rated the movie,

When they select a star rating,

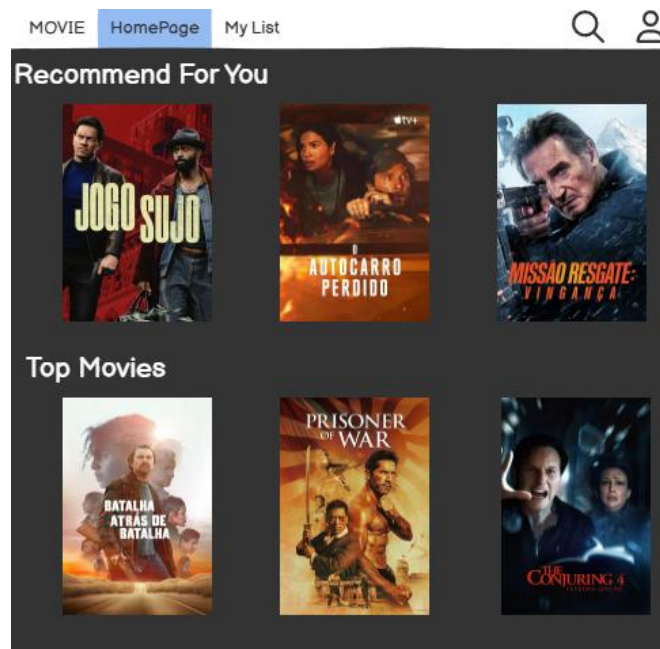
Then the rating should be saved and reflected in the movie's average rating.

[US9] Personalized Recommendations

As a user, I want to receive movie recommendations based on my rating history, so that I can discover movies I will likely enjoy.

How are we going to do this?

- UI Design: Add a "Recommended For You" section in the home/catalog page.
- Backend: Algorithm to suggest movies based on genres and ratings.
- Database: Fetch ratings and match with similar genres.
- Testing: Test recommendations with different rating patterns.



Acceptance Criteria

- Recommendation List

Given the user has rated multiple movies,

When they open the “Recommended for You” section,

Then they should see at least 5 suggestions based on their rating history and favorite genres.

- Exclude Already Rated

Given the user has rated certain movies,

When recommendations are generated,

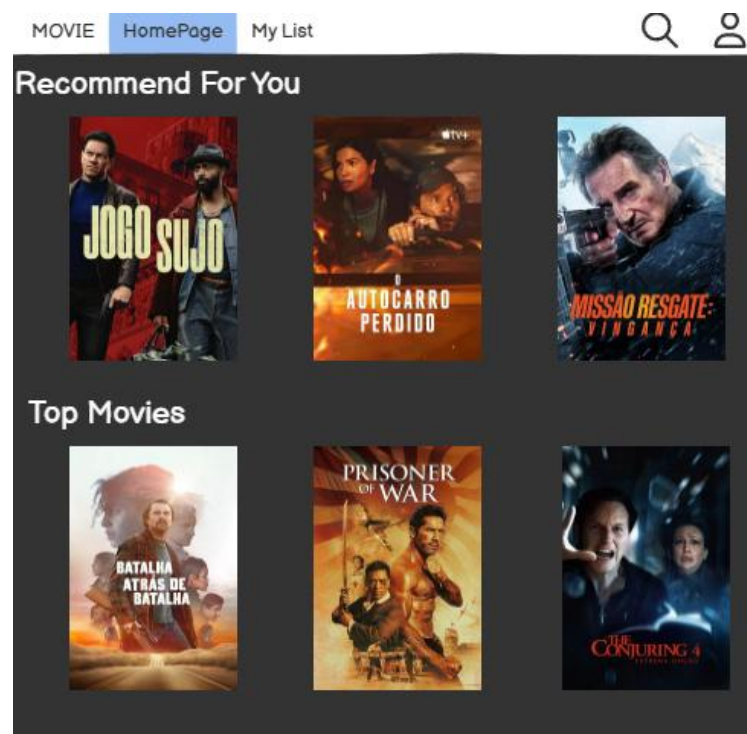
Then those movies should not appear in the recommended list

[US10] Popular Movies

As a user, I want to see a list of most popular movies, so that I can quickly find trending content.

How are we going to do this?

- UI Design: Create a “Top Movies” section.
- Backend: API to return movies sorted by number of ratings or highest average rating.
- Database: Aggregate rating counts and averages.
- Testing: Check consistency of popularity ranking.



Acceptance Criteria

- Top Movies List

Given multiple users have rated movies,

When the user opens the “Top Movies” section,

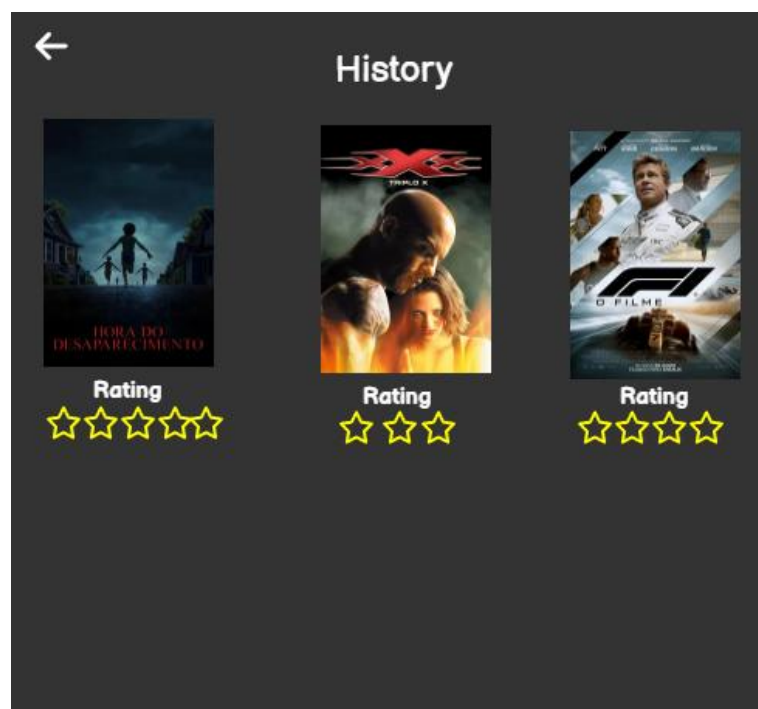
Then the top N movies with the highest ratings or rating counts should be displayed.

[US11] View History

As a user, I want to see a history of the movies I have rated, so that I can remember what I have watched.

How are we going to do this?

- UI Design: Add a “My History” page accessible from profile.
- Backend: API to return list of rated movies for the user.
- Database: Query user’s rating records.
- Testing: Ensure history updates when new ratings are added/removed.



Acceptance Criteria

- History Display

Given the user has rated movies,

When they navigate to “My History,”

Then a list of movies they have rated should be displayed with rating and date.

- Empty History

Given the user has not rated any movies,

When they open “My History,”

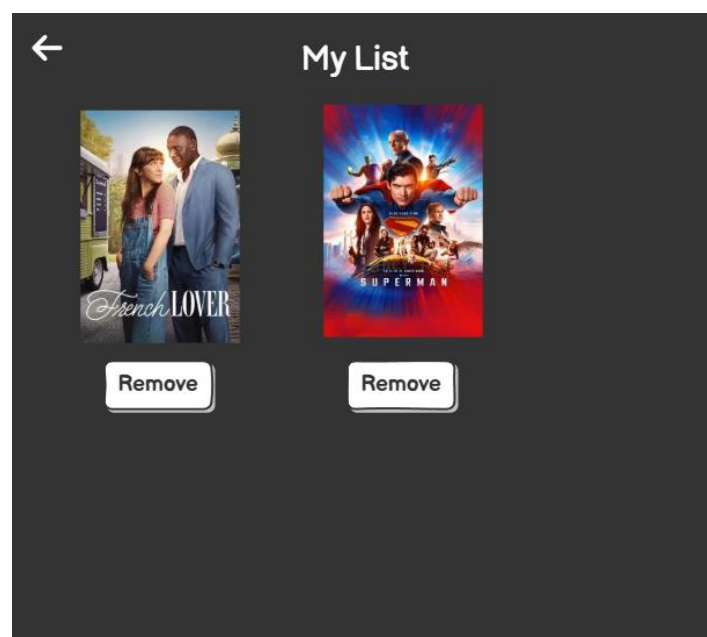
Then a message should be displayed: "No movies in your history."

[US12] Manage My List

As a user, I want to add movies to my personal watchlist (“My List”), so that I can save movies I’m interested in watching later.

How are we going to do this?

- UI Design: Add a heart icon button on each movie’s detail page. Include a dedicated “My List” page accessible from the main navigation to view all saved movies.
- Backend: Create API endpoints to: Add a movie to the user’s watchlist; Remove a movie from the watchlist; Retrieve all movies saved in the user’s watchlist.
- Database: Implement a relationship table (e.g., user_watchlist) linking user_id and movie_id.
- Error Handling: Prevent adding duplicate movies to the list and display appropriate feedback messages.
- Testing: Verify that the user can add, view, and remove movies correctly; Test for duplicates and empty list scenarios.



Acceptance Criteria

- **Add to Watchlist**

Given the user is logged in and viewing a movie,

When they click the unfilled heart icon,

Then the movie should be saved to their watchlist, and the heart should turn filled.

- **Remove from Watchlist**

Given the user has previously added a movie to their watchlist,

When they click "Remove"

Then the movie should be deleted from their watchlist and no longer appear on their "My List" page.

- **View Watchlist**

Given the user has one or more movies in their watchlist,

When they navigate to the "My List" page,

Then all saved movies should be displayed with their titles, posters, and basic info.

- **Empty Watchlist**

Given the user has not added any movies yet,

When they open "My List"

Then a message should appear: "Your list is empty. Start adding movies to watch later!"

8. Conclusion

Throughout this phase, the team defined the project's vision, scope, and requirements, ensuring a clear understanding of what the system must deliver and how it will provide value to its users.

By establishing both functional and non-functional requirements, along with use case and domain models, the team created a solid blueprint for the system's architecture and behavior. This documentation will guide the upcoming stages from implementation to testing and deployment, ensuring alignment among all team members.

The project aims not only to implement a functional recommendation system but also to integrate modern software engineering practices, including AI-assisted development. With the requirements now clearly defined, the team is ready to move into the next milestone, focusing on building and integrating the application's components following CI/CD principles.